Inheritance

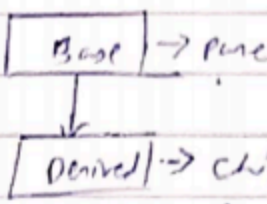
when one class (child/derived) derives the properties & methods of another class (parent/base), this promote code reusability

```
e:- class car :
        @ Static method
            def start():
                print("car started")
        @staticmethod
            def stop():
                print("car stopped")
    class Toyotacar(car):
        def __init__(self, name):
            self.name = name

car1 = Toyotacar("Fortuner")
car2 = Toyotacar("~~Benz~~ prius")

print( car1.start())
```

There are three types of inheritance

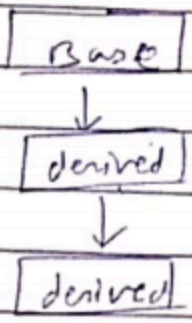
(i) is single (ii) is multi-level (iii) is Multiple

(i) single inheritance

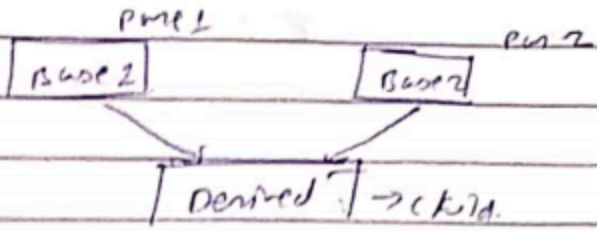


Scanned with OKEN Scanner

(ii) Multi-level



(iii) Multiple



eg:- class A:

var A = "welcome to class A"

class B:

var B = "welcome to class B"

class C(A,B)

var C = "welcome to class C"

c1 = C()

print(c1.varC)

print(c1.varB)

print(c1.varA)

Super method

superMethod is used to access methods of the parent class eg:-

class car:

def __init__(self, type):

self.type = type

```
@ Staticmethod
  def start():
      print("Car started..")
@ Staticmethod.
    def stop():
        print("Car stopped")

def Toyotacar("Car"): [obj]
      def __init__(self, name, type)
          self.name = name
          super().__init__(type).
car1 = Toyotacar("fortuner", "electonic").
print(car1.type).
```

Class method

A class method is bound to the class & receives the class as an implicit first argument.

Note :- Static method can't access or modify class state & generally for utility.

```
Class student
    @classmethod          # decorator
    def college(cls): class[obj]
        pass
```

property

we use this decorator on any method in the class to use the method as a property

polymorphism : operator overloading

when the same operator is allowed to have different meaning according to the context.

operators & Dunder functions

a + b	# addition	a.--add--(b)	Eg:- a.add(b)
a - b	# substraction	a.--sub--(b)	
a * b	# multiplication	a.--mul---(b)	
a / b	# division	a.	
a % b	# addition		

~~note:- one entity taking~~

or

The ability of a function or object to take ~~on~~ multiple forms
It allows different classes to be treated as instances of the same general class through the same interface

two type of polymorphism.

static polymorphism	dynamic polymorphism
• compile time	• run time
• method overloading	• method overriding
• execution speed is faster	• slower
• Less flexible	• Highly flexible for complex systems

Scanned with OKEN Scanner

1. Static polymorphism

it is the type of polymorphism is resolved during the compilation process. The compiler determines which method or function to call based on the parameters.

2. Dynamic polymorphism

This is resolved at runtime, it allows a program to decide which implementation of a method to execute while the code is actually running rather than when it is compiled.

• method overloading

This occurs when multiple methods in the same class have the same name but different parameters.

eg:- A math class having two add() methods one for int and one for float.

• method overriding

This happens when a subclass provide a specific implementation for a method that is already defined in its parent class.

• operator overloading

This allows a single operator to have different meanings depending on the data types it is operating on.

eg:- The + operator add two no. but concatenating two strings.